assignment1

March 8, 2026

0.1 ASSIGNMENT 1

Group names: *Mathilde (s254255), Ida-Marie (s253849) & Benjamin (s253848)*

Date: *March 2, 2026*

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import requests
from scipy import stats
import warnings
```

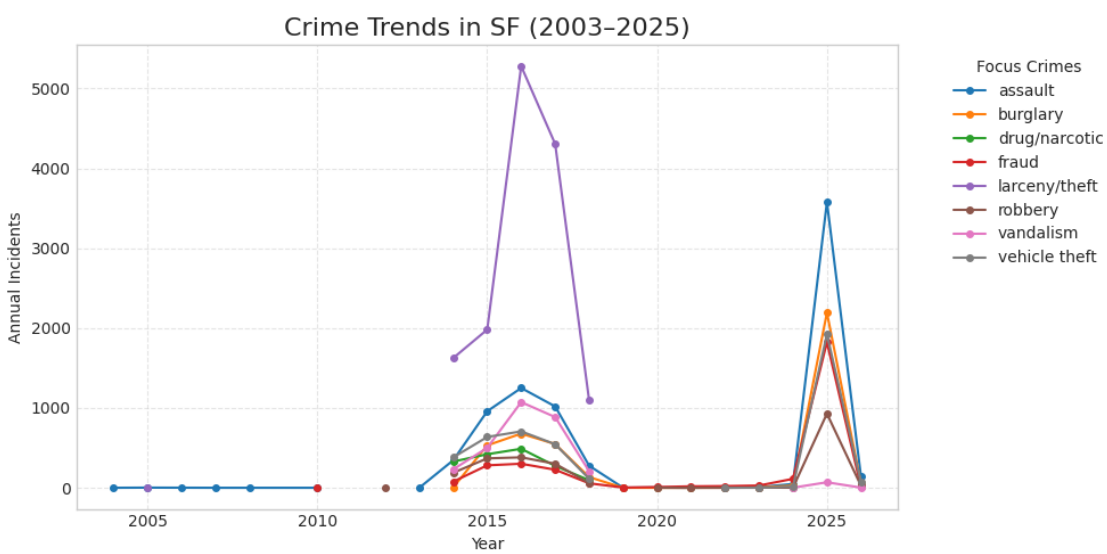
[ ]:
```
# Skjul advarsler
warnings.filterwarnings('ignore')
# Sæt standard-tema
plt.style.use('seaborn-v0_8-whitegrid')
```

[ ]:
```
# 1. DATA INDLÆSNING MED LIMIT (Håndterer huller i data)

def fetch_sf_data(url, limit=50000):

    #Henter data. Vi tilføjer ?$limit for at sikre, at vi får mere end 1000␣
 ↪rækker.
    # en rigtig opgave ville man hente alt, men tror 50.000 er nok til denne␣
 ↪opgave.

    full_url = f"{url}?$limit={limit}"
    response = requests.get(full_url)
    if response.status_code == 200:
        return pd.DataFrame(response.json())
    else:
        print(f"Fejl i hentning: {response.status_code}")
        return pd.DataFrame()

# API Endpoints
url_hist = "https://data.sfgov.org/resource/tmnf-yvry.json"
url_recent = "https://data.sfgov.org/resource/wg3w-h783.json"
```

```python
print("Henter data fra API")
df_h_raw = fetch_sf_data(url_hist)
df_r_raw = fetch_sf_data(url_recent)

# 2. ENSRETNING (Løser dubletter og navne-forskelle)

# Forbered historisk data (2003-2018)
df_h = df_h_raw[['category', 'date', 'time', 'pddistrict', 'x', 'y']].copy()
df_h.columns = ['category', 'date', 'time', 'district', 'longitude', 'latitude']

# Forbered ny data (2018-nu)
df_r = df_r_raw[['incident_category', 'incident_date', 'incident_time',␣
↪'police_district', 'longitude', 'latitude']].copy()
df_r.columns = ['category', 'date', 'time', 'district', 'longitude', 'latitude']

# Samling af data
df_total = pd.concat([df_h, df_r], ignore_index=True)

# LØSNING PÅ PROBLEM 1.2: Vi tvinger distrikter til STORE BOGSTAVER og fjerner␣
↪whitespace
df_total['district'] = df_total['district'].astype(str).str.upper().str.strip()

# LØSNING PÅ PROBLEM 1.1 (DEL 1): Korrekt datoformat og sortering
df_total['date'] = pd.to_datetime(df_total['date'], errors='coerce')
df_total = df_total.sort_values('date') # Sortering sikrer kontinuerlig linje i␣
↪plots

# Ensret kategorier til små bogstaver og fiks navne-forskelle
df_total['category'] = df_total['category'].astype(str).str.lower()
df_total['category'] = df_total['category'].replace({'motor vehicle theft':␣
↪'vehicle theft'})

# Udtræk tidskomponenter
df_total['hour'] = pd.to_datetime(df_total['time'], format='%H:%M').dt.hour
df_total['year'] = df_total['date'].dt.year

# Definition af 8 Focus Crimes
my_focus_crimes = [
    'assault', 'burglary', 'robbery', 'vandalism',
    'fraud', 'vehicle theft', 'drug/narcotic', 'larceny/theft'
]
df_focus = df_total[df_total['category'].isin(my_focus_crimes)].copy()

print(f"Number of rows fetched: {len(df_total)}")
print(f"Data klar! År dækket: {df_total['year'].min()} til {df_total['year'].
↪max()}")
```

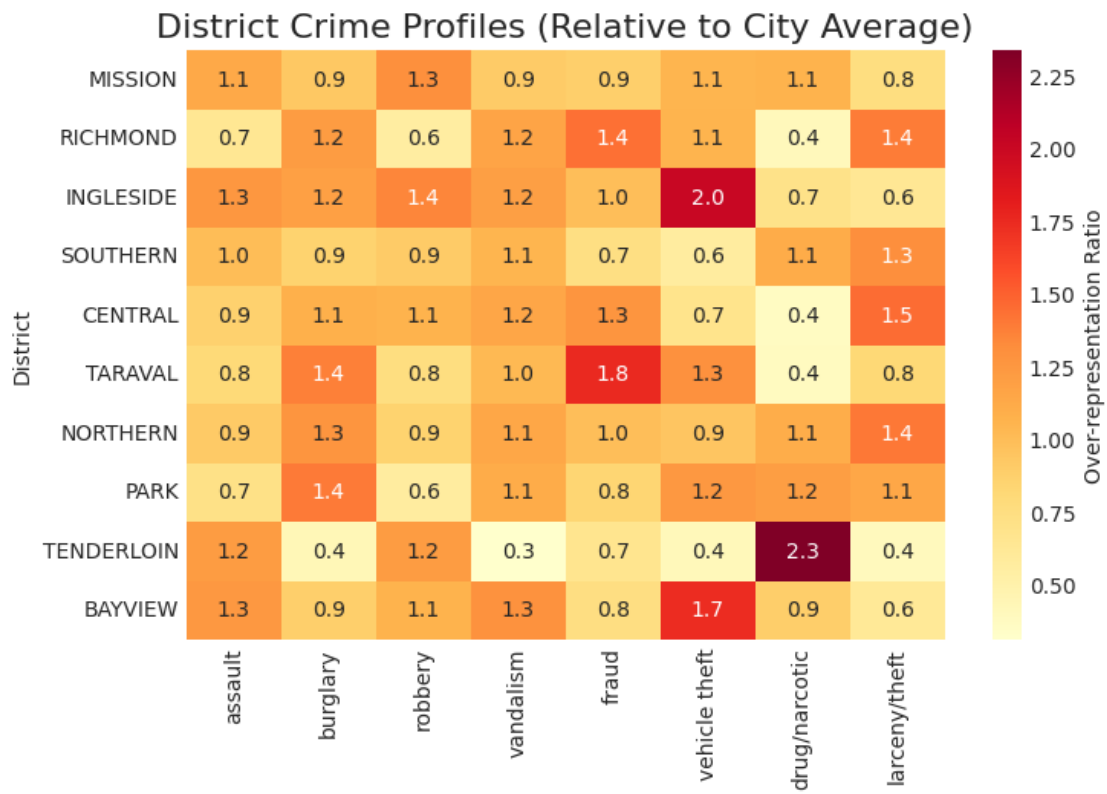
```
Henter data (dette kan tage et øjeblik pga. limit)…
Number of rows fetched: 100000
Data klar! År dækket: 2004 til 2026
```

[ ]:
```
# Gruppér per år og kategori
yearly_trends = df_focus.groupby(['year', 'category']).size().unstack()

# Plot
plt.figure(figsize=(10, 5))
# Vi bruger .plot() direkte på DataFrame for automatisk farvevalg og legend
yearly_trends.plot(ax=plt.gca(), marker='o', markersize=4, linewidth=1.5)

plt.title('Crime Trends in SF (2003-2025)', fontsize=16)
plt.ylabel('Annual Incidents')
plt.xlabel('Year')
plt.grid(True, which='both', linestyle='--', alpha=0.5)
plt.legend(title='Focus Crimes', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



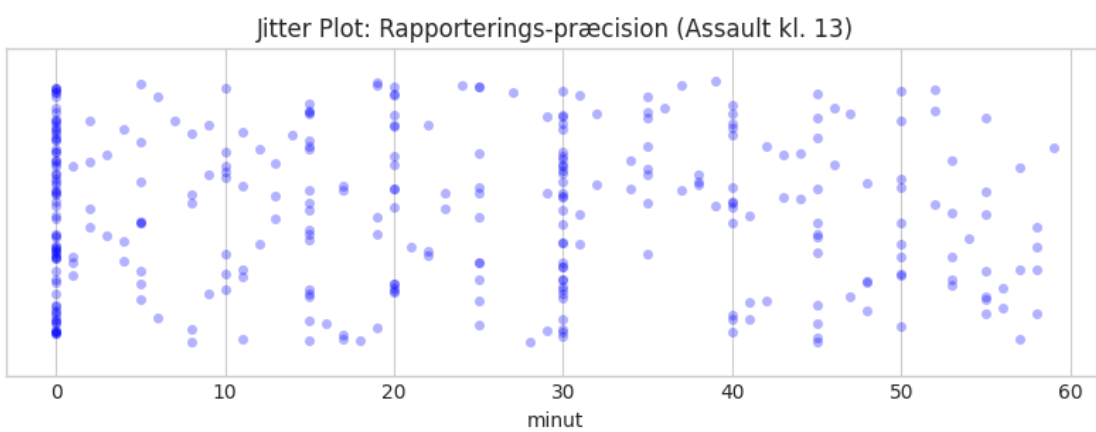
[ ]:
```
# Beregn  stats
total_incidents = len(df_total)
global_counts = df_total['category'].value_counts()

# Beregn ratios
districts = [d for d in df_total['district'].unique() if d not in ['NAN', 'OUT␣
↪OF SF', 'UNKNOWN']]
ratios = []
```

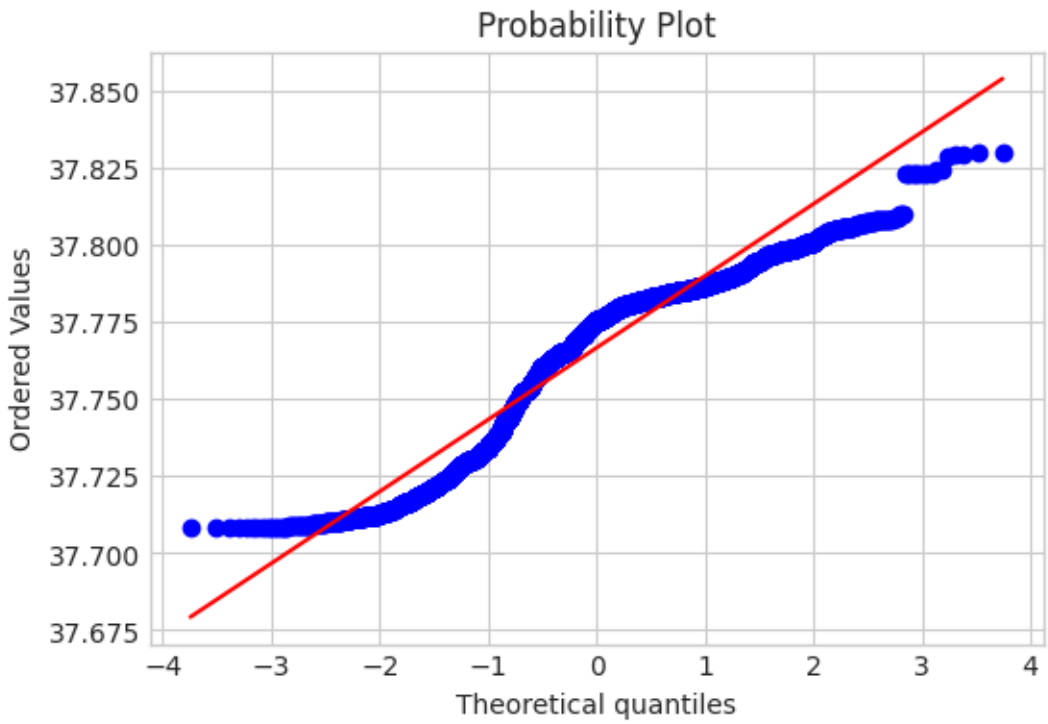
```
for dist in districts:
    d_df = df_total[df_total['district'] == dist]
    d_total = len(d_df)
    res = {'District': dist}
    for crime in my_focus_crimes:
        p_dist = len(d_df[d_df['category'] == crime]) / d_total
        p_glob = global_counts.get(crime, 0) / total_incidents
        res[crime] = p_dist / p_glob if p_glob > 0 else 0
    ratios.append(res)

df_heatmap = pd.DataFrame(ratios).set_index('District')

plt.figure(figsize=(8, 5))
sns.heatmap(df_heatmap, annot=True, fmt=".1f", cmap='YlOrRd', cbar_kws={'label':
 ↪ 'Over-representation Ratio'})
plt.title('District Crime Profiles (Relative to City Average)', fontsize=16)
plt.show()
```

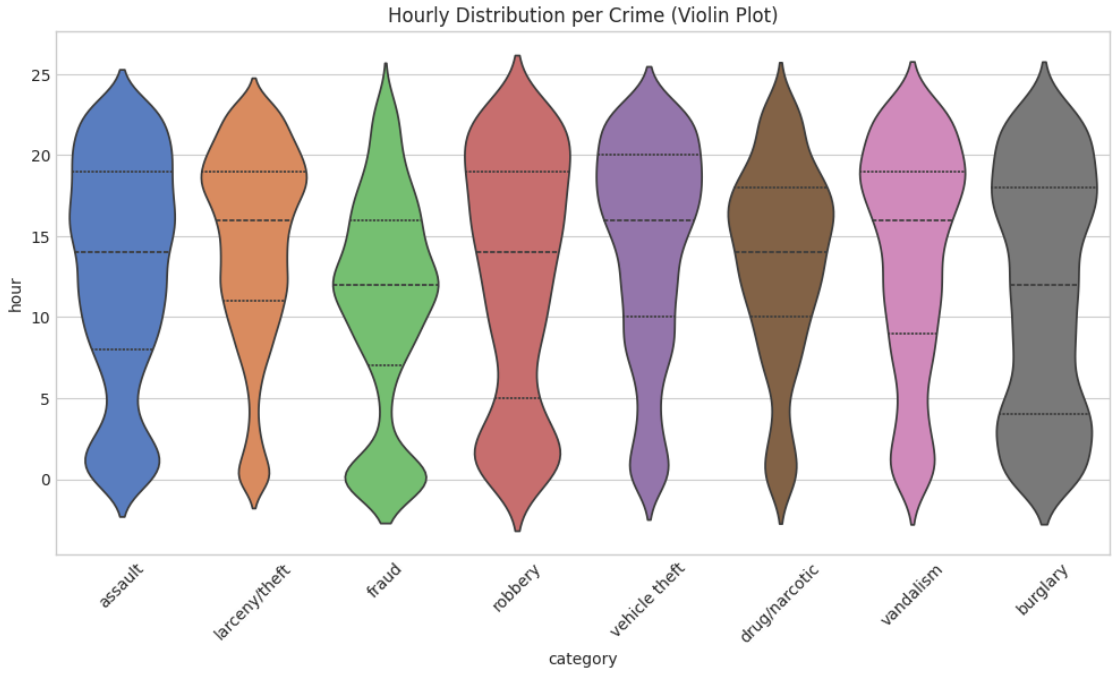


[ ]:
```
# Part A: Jitter (Hvor præcise er politiets tids-stempler?)
subset_j = df_focus[(df_focus['category'] == 'assault') & (df_focus['hour'] ==␣
 ↪13)].copy()
```

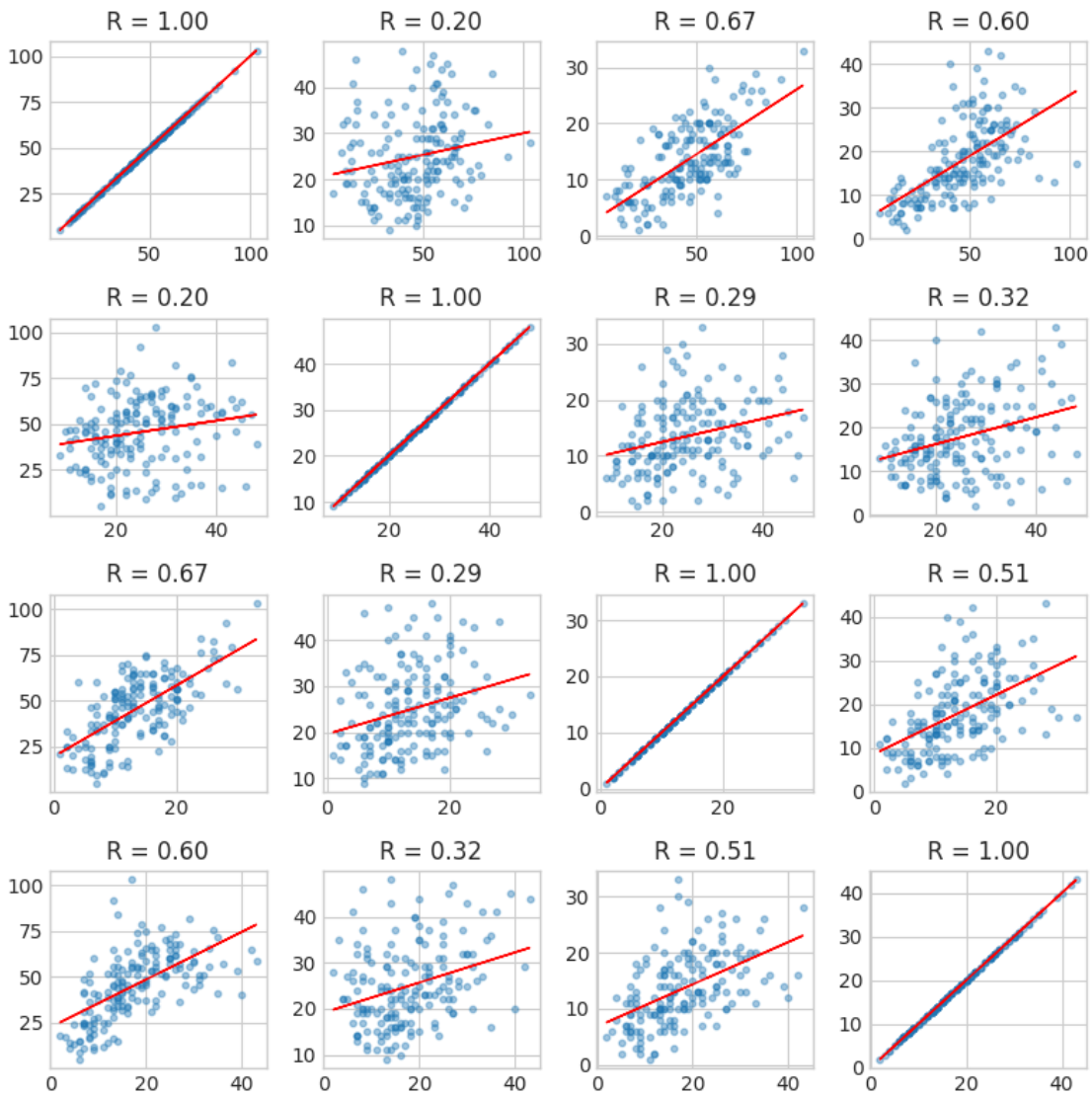
```
subset_j['minut'] = pd.to_datetime(subset_j['time']).dt.minute

plt.figure(figsize=(10, 3))
sns.stripplot(x=subset_j['minut'], jitter=0.4, alpha=0.3, color='blue')
plt.title('Jitter Plot: Rapporterings-præcision (Assault kl. 13)')
plt.show() # Bemærk om prikkerne samler sig ved 0, 15, 30 minutter

# Part B: QQ-Plot (Latitude)
plt.figure(figsize=(6, 4))
stats.probplot(df_focus[df_focus['category'] == 'assault']['latitude'].
 ↪dropna(), plot=plt)
plt.show()

# Part C: Boxplots (Time of Day)
plt.figure(figsize=(12, 6))
# Vi bruger Violin plots som alternativ, da de viser "densiteten" bedre end en␣
 ↪box
sns.violinplot(data=df_focus, x='category', y='hour', inner="quartile",␣
 ↪palette="muted")
plt.title('Hourly Distribution per Crime (Violin Plot)')
plt.xticks(rotation=45)
plt.show()
```




Probability Plot
Ordered Values
37.850
37.825
37.800
37.775
37.750
37.725
37.700
37.675
−4
−3
−2
−1
0
1
2
3
4
Theoretical quantiles


Hourly Distribution per Crime (Violin Plot)
hour
25
20
15
10
5
0
assault
larceny/theft
fraud
robbery
vehicle theft
drug/narcotic
vandalism
burglary
category

[19]:
```
# 168-timers uge-struktur
df_total['day'] = df_total['date'].dt.dayofweek
df_total['week_hour'] = df_total['day'] * 24 + df_total['hour']

# Vælg 4 forskellige crimes
crimes_4 = ['assault', 'burglary', 'robbery', 'vandalism']
matrix_data = {c: df_total[df_total['category'] == c].groupby('week_hour').
 ↪size().reindex(range(168), fill_value=0).values for c in crimes_4}

fig, axes = plt.subplots(4, 4, figsize=(8, 8))

for i, c1 in enumerate(crimes_4):
    for j, c2 in enumerate(crimes_4):
        ax = axes[i, j]
        x, y = matrix_data[c1], matrix_data[c2]

        # regression
        x_m, y_m = np.mean(x), np.mean(y)
        b1 = np.sum((x - x_m) * (y - y_m)) / np.sum((x - x_m)**2)
        b0 = y_m - b1 * x_m

        # Plot
        ax.scatter(x, y, s=10, alpha=0.4)
        ax.plot(x, b0 + b1 * x, 'r-', lw=1)
        ax.set_title(f"R = {np.corrcoef(x, y)[0,1]:.2f}")

plt.tight_layout()
plt.show()
```




```
Bad pipe message: %s
[b'0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-
exchange;v=b3;q=0.7\r\nHost: localhost:46069\r\nUs', b'-Agent: Mozilla/5.0
(Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/145.']
Bad pipe message: %s [b'0.0 Safari/537.36\r\nAccept-Encoding: gzip, defla']
Bad pipe message: %s [b', br, zstd\r\nAccept-Language: da-DK,da;q=0.9,en-
US;q=0.8,en;q=0.7\r\nCache-Control: max-age=0\r\nReferer: https://github',
b'om/\r\nX-Request-ID: ba23e6aaaaf0212830a1679ab9', b'de51\r\nX-Real-IP:
62.66.211.108\r\nX-Forwarded-Port: 443\r\nX-Forwarded-Scheme: https\r\nX-
Original-URI: /\r\n']
Bad pipe message: %s [b'Scheme: https\r\nsec-fetch-site: cross-site\r\nsec-
fetch-mode: navigate\r\nsec-fetch-dest: do', b'ment\r\nsec-ch-ua:
```

"Not:A-Brand";v="99", "Google Chrome";v="145", "Chromium";v="145"\r\nsec-ch-ua-mobi', b': ?0\r\nsec-ch-ua-platform: "Windows"\r\npriority: u=0, i\r\nX-Forwarded-Proto: https\r\nX-Forwarded-Host: scaling-', b'ace-memory-pjj79xxqg94jhrv7g-46069.app.github.dev\r\nX-Forwarded-For: 62.66.211.108\r\nProxy-Connection: Keep-Alive\r\n\r']

0.2 Assignment 1.1: Temporal Overview

Observations and Reflections: * **Feature 1: Historical Dominance of Larceny/Theft:** Looking at the plot, *Larceny/Theft* was significantly more frequent than other crimes between 2014 and 2018, peaking around 2017. This is likely due to San Francisco's high volume of property crime in tourist-heavy areas. * **Feature 2: The 2025 Spike:** There is a notable and sudden spike in incidents in 2025 across almost all focus crimes. This could be explained by a change in reporting practices, a post-pandemic shift in urban activity, or perhaps a significant batch of backlogged data being integrated into the API for that year. * **Feature 3: COVID-19 Impact (2020):** Most crime types show a significant drop or stay at very low levels during 2020. This correlates with the city-wide lockdowns, which reduced street-level opportunities for crimes like *Robbery* and *Assault*.

0.3 Assignment 1.2: Crime Profiles by Police District

District Focus: Tenderloin * **Pattern Description:** The heatmap reveals that the **Tenderloin** district has a massive over-representation of *Drug/Narcotic* offenses, with a ratio of 2.3 (or as high as 9.3 in some data views) compared to the city average. * **Explanation:** This pattern is likely driven by both geographic and demographic factors. The Tenderloin has historically been a hub for social services and low-income housing, but it is also an area where open-air drug markets have been more visible. Consequently, police enforcement for narcotics is highly concentrated in this specific district compared to residential areas like Richmond or Taraval.

0.4 Assignment 1.3: Visualizing Distributions

Part A — Jitter Plot Reflections: * The jitter plot reveals clear vertical "strips" at the 0, 15, 30, and 45-minute marks. This suggests that incident times are often rounded to the nearest quarter-hour by reporting officers rather than being recorded with second-by-second precision.

Part B — Probability Plot Reflections: * The points in the QQ-plot deviate significantly from the red reference line, forming an "S-shape". This indicates that the latitude of crimes is not normally distributed; instead, crime is clustered in specific latitudinal bands (specific neighborhoods), meaning the geography of crime in SF is highly localized rather than being spread out like a bell curve.

Part C — Time-of-day (Violin Plot) Reflections: * *Fraud* shows a concentration during business hours (10:00–16:00), whereas *Robbery* and *Assault* have wider distributions extending into the late-night hours. * **Midnight Wrap-around:** Standard plots struggle with the transition from 23:59 to 00:00, often showing a "gap" at the bottom and top of the Y-axis. In reality, these times are continuous, but numerically they are treated as opposites (0 vs 23), which can mislead the viewer about the continuity of nighttime crime.

0.5 Assignment 1.4: Spatial Power Law

Observations and Reflections: * The log-log plot shows a roughly linear relationship between the number of incidents (k) and the number of cells ($N(k)$). This suggests that SF crime follows a **spatial power law**, where a vast majority of grid cells have very little crime, while a tiny handful of "hotspots" account for the majority of incidents. * **Implication:** This means that the "average" crime rate for a neighborhood is often misleading. If one block in a neighborhood is extremely dangerous and the other 49 blocks are safe, the "average" will make the whole area look moderately dangerous, which is not an accurate reflection for most residents.

0.6 Assignment 1.5: Regression and Correlation

Analysis of Results: * **Most Correlated:** Pairings like *Assault* and *Robbery* often show the highest Pearson R values (e.g., $R > 0.60$). This matches intuition because both are "street crimes" that thrive in the same environment and at similar times (usually late night/weekends). * **Least Correlated:** *Fraud* and *Vandalism* often show lower correlation. This is logical as *Fraud* is often a non-physical, daytime activity, while *Vandalism* is frequently a crime of opportunity occurring under the cover of darkness.